

Experiment 6 — Valid Identifier

Develop a lexical Analyzer to test whether a given identifier is valid or not using C.”

Algorithm

1. Start.
2. Read the identifier.
3. Check whether the first character is an alphabet.
4. Check remaining characters for alphabets or digits.
5. If all conditions are satisfied, print **Valid identifier**.
6. Otherwise, print **Invalid identifier**.
7. Stop.

```
#include <stdio.h>
#include <ctype.h>
```

```
int main()
{
    char id[20];
    int i;
    int valid=1;

    printf("Enter the identifier:\n");
    scanf("%s",id);
    if(!isalpha(id[0]))
    {
        valid=0;
    }
    else
    {
        for(i=1;id[i]!='\0';i++)
        {
            if(!isalpha(id[i])&&!isdigit(id[i]))
            {
                valid=0;
                break;
            }
        }
    }
    if(valid==1)
    {
        printf("Valid identifier");
    }
    else
    {
        return 0;
    }
}
```

```
Enter the identifier:
abc123
Valid identifier
```

Program 5 — Count whitespaces and newline characters using C

Algorithm

1. Start the program.
2. Read the input text until ~.
3. Check every character one by one.
4. If it is a space or tab, increase the space count.
5. If it is a newline, increase the line count.
6. Otherwise, increase the character count.
7. Display the results.
8. Stop.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      char str[100];
6      int spaces = 0;
7      int lines = 0;
8      int characters = 0;
9      int i;
10
11     printf("Enter text and use ~ to stop:\n");
12     scanf("%[^\n]", str);
13
14     for(i = 0; str[i] != '\0'; i++)
15     {
16         if(str[i] == ' ' || str[i] == '\t')
17         {
18             spaces++;
19         }
20         else if(str[i] == '\n')
21         {
22             lines++;
23         }
24         else
25         {
26             characters++;
27         }
28     }
29
30     if(characters > 0)
31     {
32         lines++;
33     }
34
35     printf("\nNumber of spaces: %d", spaces);
36     printf("\nNumber of lines: %d", lines);
37     printf("\nNumber of characters: %d", characters);
38
39     return 0;
40 }
```

Enter text and use ~ to stop:
Hello World
Compiler Design~

Number of spaces: 2
Number of lines: 2
Number of characters: 24

Program 2 — Check whether a line is a comment or not

Algorithm

1. Start.
2. Read the input line.
3. Check whether it begins with //.
4. If yes, print It is a comment.
5. Otherwise check whether it begins with /* and contains */.
6. If yes, print It is a comment.
7. Otherwise print It is not a comment.
8. Stop.

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main()
5  {
6      char str[100];
7
8      printf("Enter a line: ");
9      fgets(str, sizeof(str), stdin);
10
11     if(str[0] == '/' && str[1] == '/')
12     {
13         printf("It is a comment");
14     }
15     else if(str[0] == '/' && str[1] == '*')
16     {
17         if(strstr(str, "*/") != NULL)
18         {
19             printf("It is a comment");
20         }
21         else
22         {
23             printf("It is not a comment");
24         }
25     }
26     else
27     {
28         printf("It is not a comment");
29     }
30
31     return 0;
32 }
```

Enter a line: //hello
It is a comment

Program 4 — Validate Operators Using C

Algorithm

1. Start.
2. Read an operator from the user.
3. Use switch to check the entered operator.
4. If it is +, print Addition.
5. If it is -, print Subtraction.
6. If it is *, print Multiplication.
7. If it is /, print Division.
8. If it is %, print Modulus.
9. Otherwise, print Not a valid operator.
10. Stop.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      char op;
6
7      printf("Enter an operator: ");
8      scanf("%c", &op);
9
10     switch(op)
11     {
12         case '+':
13             printf("Addition operator");
14             break;
15
16         case '-':
17             printf("Subtraction operator");
18             break;
19
20         case '*':
21             printf("Multiplication operator");
22             break;
23
24         case '/':
25             printf("Division operator");
26             break;
27
28         case '%':
29             printf("Modulus operator");
30             break;
31
32         default:
33             printf("Not a valid operator");
34     }
35
36     return 0;
37 }
```

Enter an operator: +
Addition operator

Program 17 — Count Characters, Words and Lines

Algorithm

1. Start.
2. Read the input text until ~.
3. Check every character one by one.
4. If a space is found, increase the word count.
5. If a newline is found, increase both line and word counts.
6. Otherwise, increase the character count.
7. Add one final word and one final line.
8. Display words, lines, and characters.
9. Stop.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      char str[100];
6      int words = 0;
7      int lines = 0;
8      int characters = 0;
9      int i;
10
11     printf("Enter the text and use ~ to stop:\n");
12     scanf("%[^\n]", str);
13
14     for(i = 0; str[i] != '\0'; i++)
15     {
16         if(str[i] == ' ')
17         {
18             words++;
19         }
20         else if(str[i] == '\n')
21         {
22             lines++;
23             words++;
24         }
25         else
26         {
27             characters++;
28         }
29     }
30
31     if(characters > 0)
32     {
33         words++;
34         lines++;
35     }
36
37     printf("\nTotal number of words: %d", words);
38     printf("\nTotal number of lines: %d", lines);
39     printf("\nTotal number of characters: %d", characters);
40
41     return 0;
42 }
```

Enter the text and use ~ to stop:

```
void main()
{
    int a;
    int b;
}~
```

Total number of words: 8

Total number of lines: 5

Total number of characters: 22

